



Final Design Submission

Advanced Web Development

Submitted on: 27.01.2026

Submitted By:

Pawan Lamsal

1751362

lamsal@uni-hildesheim.de

Sher Afgan

1751148

afgan@uni-hildesheim.de

Suman Neupane

1751373

neupane@uni-hildesheim.de

Student Dorm Management System

The Student Dormitory Management System, a web-based application intended to support and digitalize core administrative processes in student dormitories. The system targets two user roles, students and dormitory administrators, and defines functionality for online dormitory applications, tenant profile management, digital contract handling, rent payment record management, and complaint submission and processing. The requirements include mechanisms for uploading and downloading documents such as contracts and rent payment receipts, as well as a backend process for generating rent payment history reports as a longer-running computation with progress feedback displayed on the frontend. In addition, the system provides role-based access control, dialog-based user confirmations, and router-based menu navigation.

Structural Overview – Class Diagram

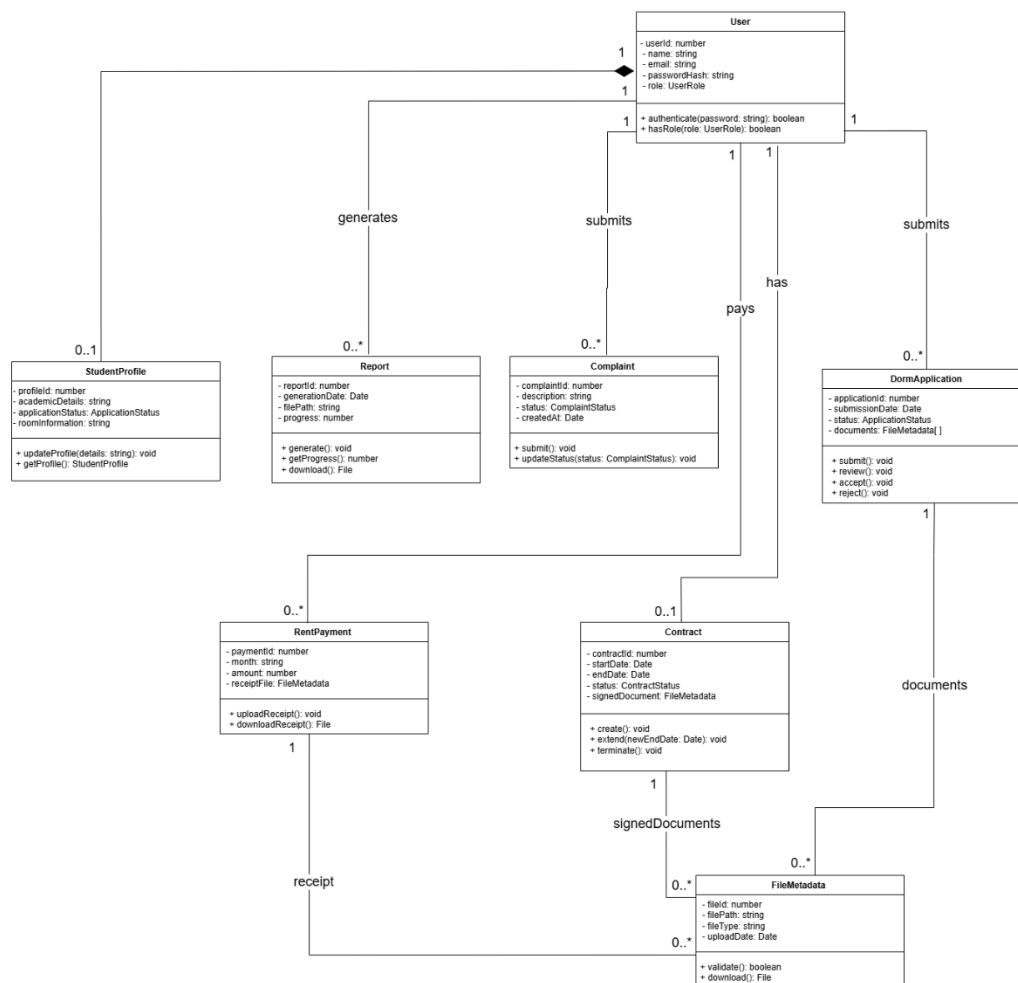


Figure 1: Domain Level Class Diagram

Student and administrator users are modeled using roles within the User entity. Student-specific data is modeled via composition using StudentProfile. Angular components interact exclusively through services, guarded by router guards and interceptors. Inheritance is intentionally not used.

Angular Architecture Components, Services and Modules

<<ngComponent>>: ngComponents are responsible for managing the user interface and interactions which are listed as follows:

- **AppComponent:** Serves as the root component of the Angular application and hosts the main router outlet and global layout structure.
- **NavMenuComponent:** Provides menu-based navigation using Angular routing and adapts visible menu options based on the authenticated user's role.
- **LoginComponent:** Manages user authentication and routes users to the appropriate dashboard based on their assigned role.
- **StudentDashboardComponent:** Provides students with an overview of their application status, contract status, complaint status, and access to document handling and report generation features.
- **AdminDashboardComponent:** Enables administrators to view and manage pending dormitory applications, handle complaints, manage contracts, and initiate report generation.
- **ApplicationComponent:** Handles dormitory application form input, validation, and submission by students.
- **ContractComponent:** Manages the display of contract information and supports contract-related actions such as viewing, extension, termination, and signed contract uploads.
- **ComplaintComponent:** Manages complaint submission by students and supports complaint status updates by administrators.
- **DocumentsComponent:** Handles the upload and download of required documents such as contracts and supporting application files.
- **ReceiptComponent:** Manages the upload and download of monthly rent payment receipts for students.
- **ProgressComponent:** Displays progress information and visual indicators for long-running backend tasks such as rent payment report generation.

<<ngService>>: ngServices encapsulate business logic and handle interactions between Angular components and backend data sources.

- **AuthenticationService:** Handles user authentication, session management, and role-based authorization logic for students and administrators.
- **ApplicationService:** Processes dormitory application data, supports application submission, review, acceptance, and rejection, and persists application records in the backend.
- **ContractService:** Manages dormitory contract lifecycle operations, including creation, extension, termination, and retrieval of contract information.
- **ComplaintService:** Handles complaint creation, retrieval, status updates, and complaint management operations.
- **ReceiptService:** Manages rent payment records and coordinates the upload, retrieval, and storage of rent payment receipts.
- **FileService:** Provides generic file upload and download functionality for contracts, application documents, and rent payment receipts.
- **ReportService:** Generates rent payment history reports as a long-running backend task and prepares generated reports for download.

- **ProgressService**: Tracks execution progress of long-running backend computations and exposes progress information to the frontend.

Angular Infrastructure Services (<<ngService>>): Infrastructure ngServices provide application-wide technical support such as authentication, authorization, and route protection.

- **AuthGuard**: Protects application routes by ensuring that only authenticated users can access secured views.
- **RoleGuard**: Enforces role-based access control by restricting routes to authorized user roles (Student or Admin).

<<ngModule>>: ngModules structure the Angular application by grouping related components, services, and routing logic into cohesive functional units.

- **AppModule**: The root Angular module responsible for bootstrapping the application and registering core components and services.
- **CoreModule**: Provides singleton services such as authentication, guards, and interceptors shared across the application.
- **StudentModule**: Encapsulates student-specific components and functionality, including dashboards, applications, documents, and reports.
- **AdminModule**: Encapsulates administrator-specific components and functionality, including application review, complaint management, and contract administration.

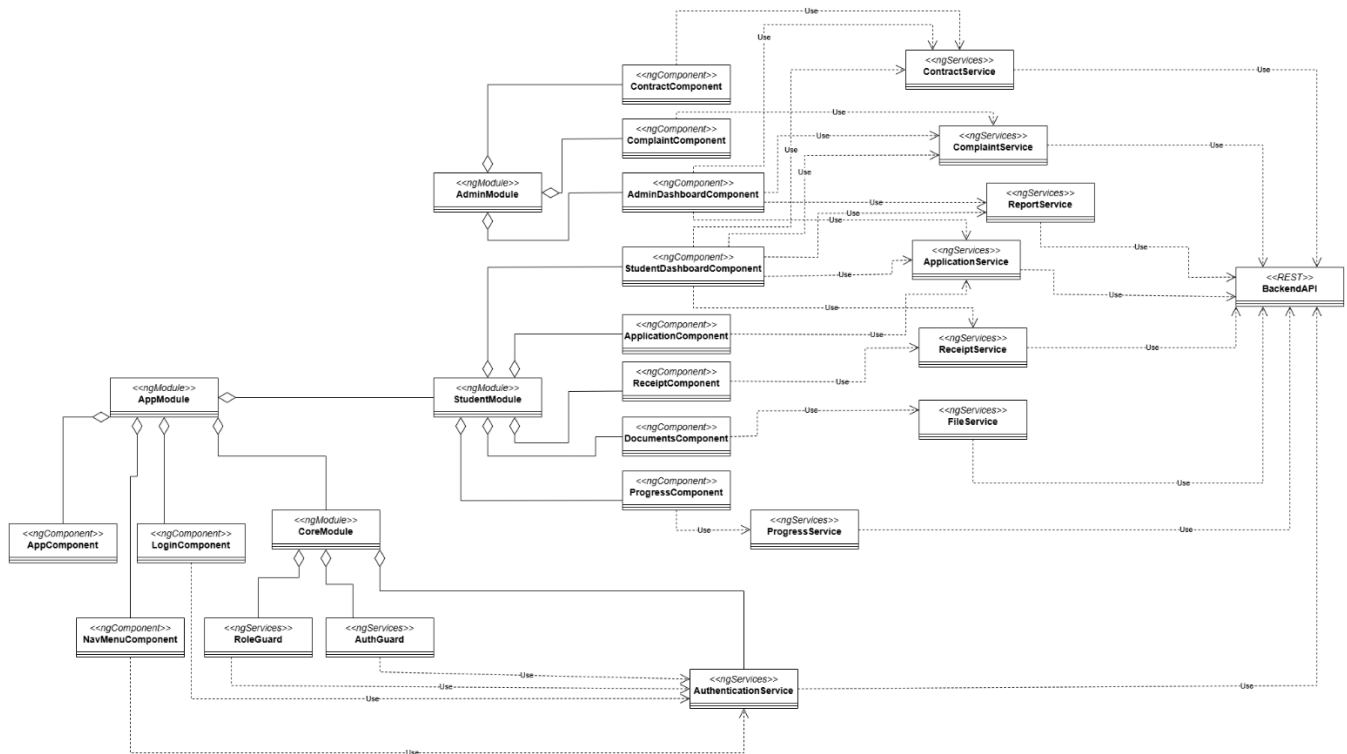


Figure 2: Architecture Level Class Diagram

Data

This section describes the core data structures of the Student Dorm Management System and the relationships between entities based on a role-based user model and composition-oriented design.

- **Entities and Attributes**

- **User**

- **userId**: Unique identifier for the user account.
 - **name**: Full name of the user.
 - **email**: User's email address used for authentication.
 - **passwordHash**: Encrypted password for secure authentication.
 - **role**: Role assigned to the user (STUDENT or ADMIN).

- **StudentProfile**

- **profileId**: Unique identifier for the student profile.
 - **userId**: Reference to the associated user account.
 - **academicDetails**: Academic and personal information of the student.
 - **applicationStatus**: Current dormitory application status.
 - **roomInformation**: Assigned room details, if applicable.

- **DormApplication**

- **applicationId**: Unique identifier for the dormitory application.
 - **userId**: Reference to the user submitting the application (role = STUDENT).
 - **submissionDate**: Date of application submission.
 - **status**: Application status (Pending, Accepted, Rejected).
 - **documents**: Uploaded supporting application documents.

- **Contract**

- **contractId**: Unique identifier for the dormitory contract.
 - **userId**: Reference to the associated student user.
 - **startDate**: Contract start date.
 - **endDate**: Contract end date.
 - **status**: Contract status (Active, Extended, Terminated).
 - **signedDocument**: Uploaded signed contract file.

- **RentPayment**

- **paymentId**: Unique identifier for the rent payment record.
 - **userId**: Reference to the associated student user.
 - **month**: Payment month.
 - **amount**: Rent amount paid.
 - **receiptFile**: Uploaded rent payment receipt.

- **Complaint**

- **complaintId**: Unique identifier for the complaint.
 - **userId**: Reference to the student user who submitted the complaint.
 - **description**: Complaint details.
 - **status**: Complaint status (Submitted, In Progress, Resolved).
 - **createdAt**: Date of complaint submission.

- **Report**

- **reportId**: Unique identifier for the generated report.
 - **generatedBy**: Reference to the user who initiated the report generation.

- **generationDate**: Date of report creation.
 - **filePath**: Storage location of the generated report file.
- **FileMetadata**
 - **fileId**: Unique identifier for an uploaded file.
 - **filePath**: Storage path of the file.
 - **fileType**: Type of the file (e.g., PDF).
 - **uploadDate**: Timestamp of file upload.
- **Relationships**
 - A **User** has exactly one role (STUDENT or ADMIN).
 - A **User** with role STUDENT may have **one StudentProfile** (1-to-0..1).
 - A **User** with role STUDENT can submit **multiple DormApplications** (1-to-Many).
 - A **User** with role STUDENT can have **at most one active Contract** at a time (1-to-0..1).
 - A **User** with role STUDENT can have **multiple RentPayments** (1-to-Many).
 - A **User** with role STUDENT can submit **multiple Complaints** (1-to-Many).
 - A **User** can generate **multiple Reports** (1-to-Many).
 - A **DormApplication**, **Contract**, and **RentPayment** can reference one or more **FileMetadata** entries for uploaded documents.

Behavioral Overview

● Actors and Interactions

Actors

- **Student**: Interacts with the system to register, log in, submit dormitory applications, upload required documents, view application and contract status, submit complaints, upload rent payment receipts, and generate rent payment history reports.
- **Admin**: Uses the system to log in, review and decide on dormitory applications, manage student complaints, administer dormitory contracts (create, extend, terminate), and generate rent payment history reports.

Backend & Persistence

- **Database**: Stores user accounts, student profiles, dormitory applications, contracts, rent payment records, complaints, file metadata, and generated report information.

● Interaction Flow

- User interactions are initiated through Angular components (e.g., LoginComponent, StudentDashboardComponent, AdminDashboardComponent), which handle user input, navigation, and presentation logic.
- Angular components delegate business operations (such as application submission, complaint handling, contract management, file uploads, and report generation) to dedicated Angular services to ensure a clear separation of concerns.
- Angular services (e.g., ApplicationService, ContractService, ComplaintService, FileService, ReceiptService) communicate with backend REST APIs to process requests and persist or retrieve data.
- Authentication and authorization are enforced using AuthenticationService in combination with route guards (AuthGuard, RoleGuard) to ensure role-based access control for students and administrators.
- Backend services interact with the database and file storage to manage user accounts,

- applications, contracts, complaints, payment records, and uploaded documents.
- Long-running backend operations, such as rent payment history report generation, are executed asynchronously by the backend and expose execution progress.
- Progress updates are retrieved via ProgressService and displayed to the user through the ProgressComponent, providing visual feedback during backend processing.

• Sequence Diagrams

Each of the following sequence diagram represents a concrete scenario of a use case and illustrates the interaction between actors, Angular components, and backend services.

• Login Scenario

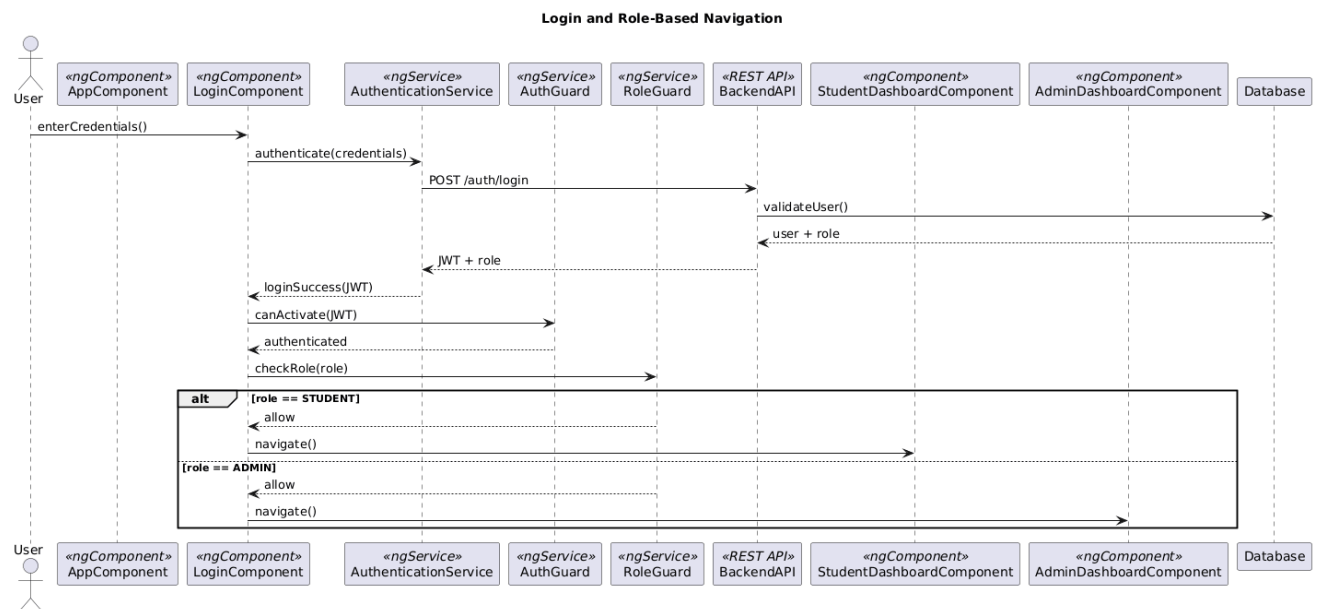


Figure 3: Login Sequence Diagram

- The user enters credentials via the LoginComponent.
- The component forwards the credentials to the AuthenticationService.
- The service sends the authentication request to the backend for validation.
- The backend validates the credentials against stored user data.
- Based on the returned role (Student or Administrator), route access is checked using AuthGuard and RoleGuard.
- The user is redirected to the corresponding dashboard (StudentDashboardComponent or AdminDashboardComponent).

- **Application Submission Scenario**

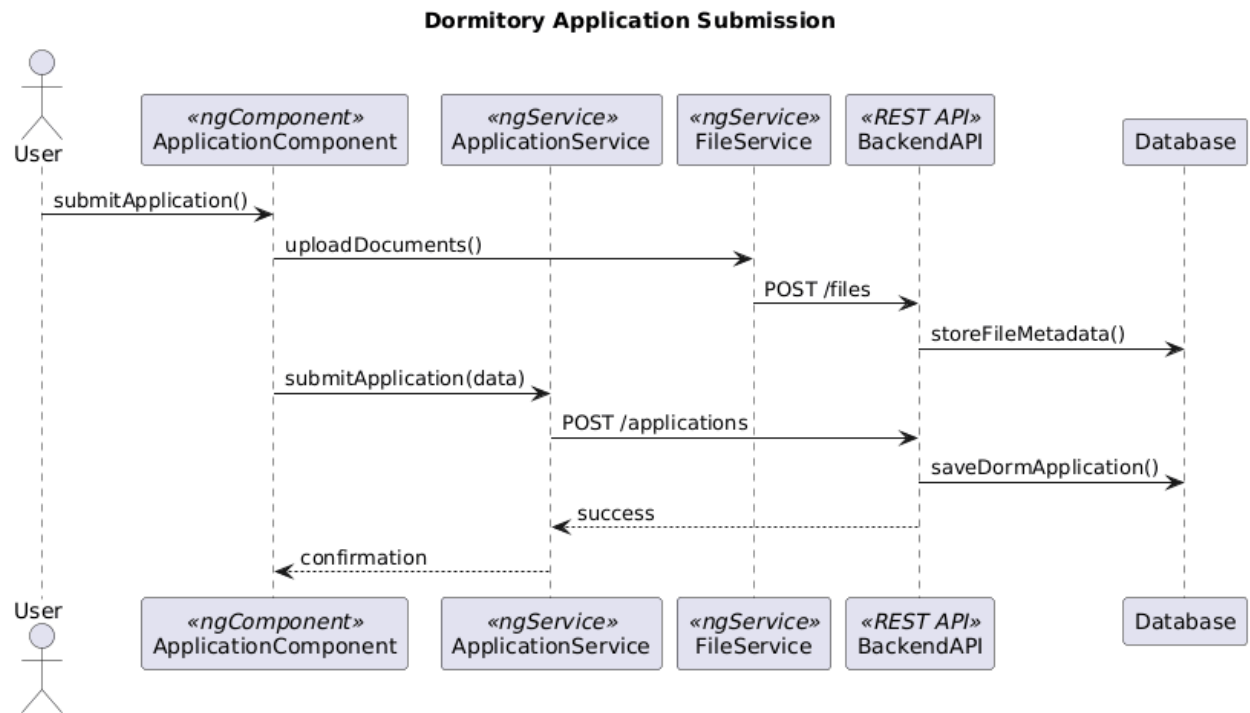


Figure 4: Application Submission Sequence Diagram

- The student fills out and submits the dormitory application via the ApplicationComponent.
- The component delegates document uploads to the FileService.
- Uploaded files are stored by the backend and associated metadata is persisted.
- The application data is forwarded to the ApplicationService.
- The backend stores the dormitory application data.
- A confirmation is returned and displayed to the student.

- **Application Review & Decision Scenario**

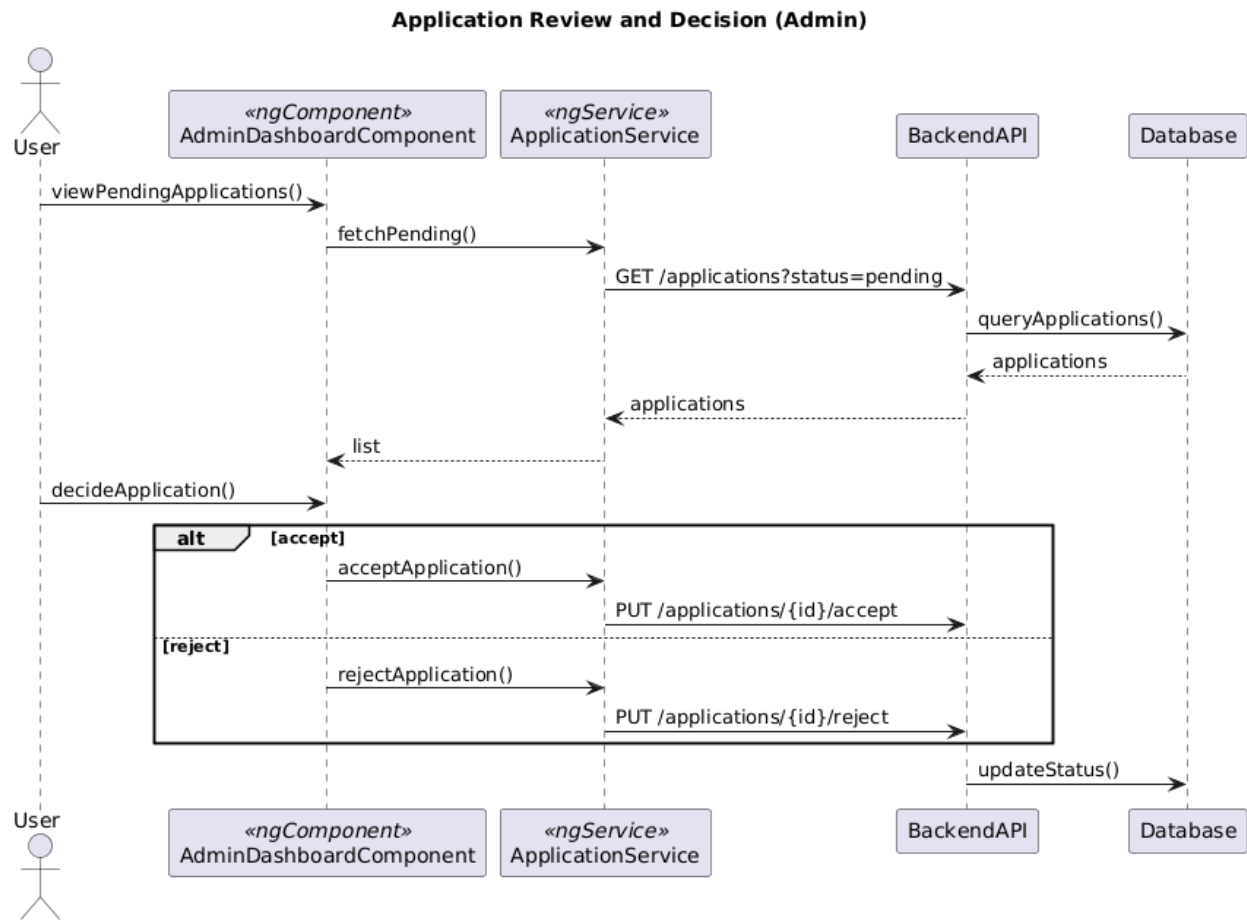


Figure 5: Application Review & Decision Sequence Diagram

- The administrator accesses pending applications through the AdminDashboardComponent.
- The component requests application data from the ApplicationService.
- The backend retrieves pending applications from the database.
- The administrator accepts or rejects an application via the dashboard.
- The decision is processed by the ApplicationService and persisted by the backend.

- Complaint Submission & Processing Scenario

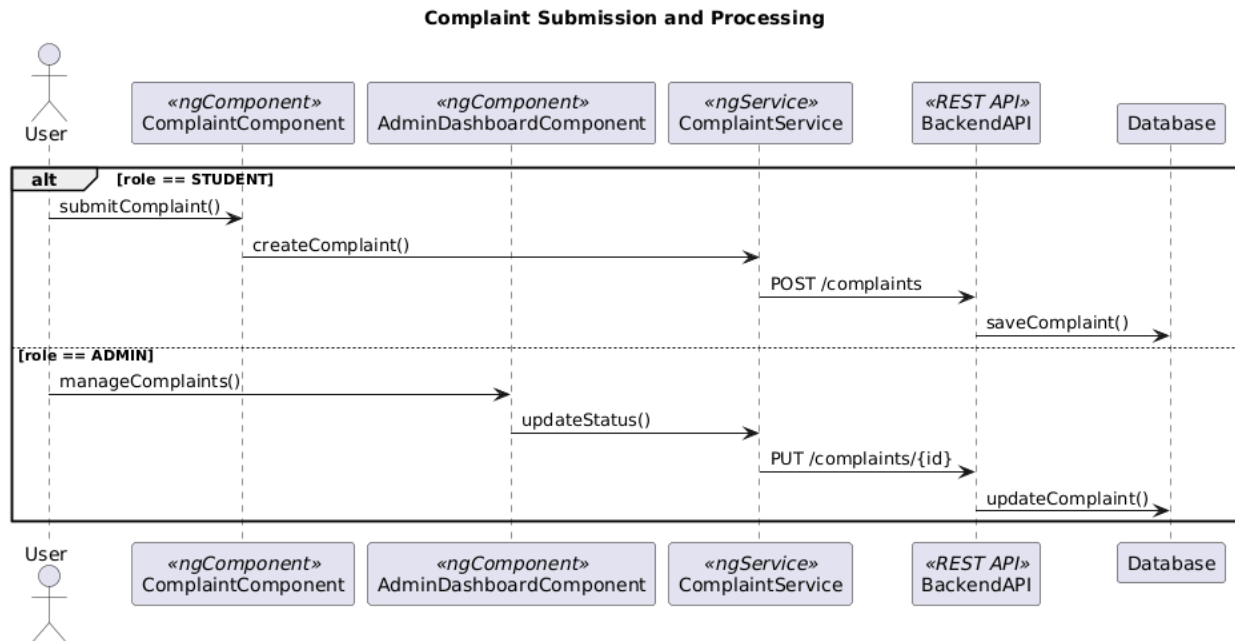


Figure 6: Complaint Submission & Processing Sequence Diagram

- A student submits a complaint using the ComplaintComponent.
- The component forwards the complaint data to the ComplaintService.
- The backend stores the complaint in the database.
- Administrators retrieve and manage complaints via the AdminDashboardComponent.
- Complaint status updates are processed by the ComplaintService and stored by the backend.

- **Rent Payment Receipt Upload & Download Scenario**

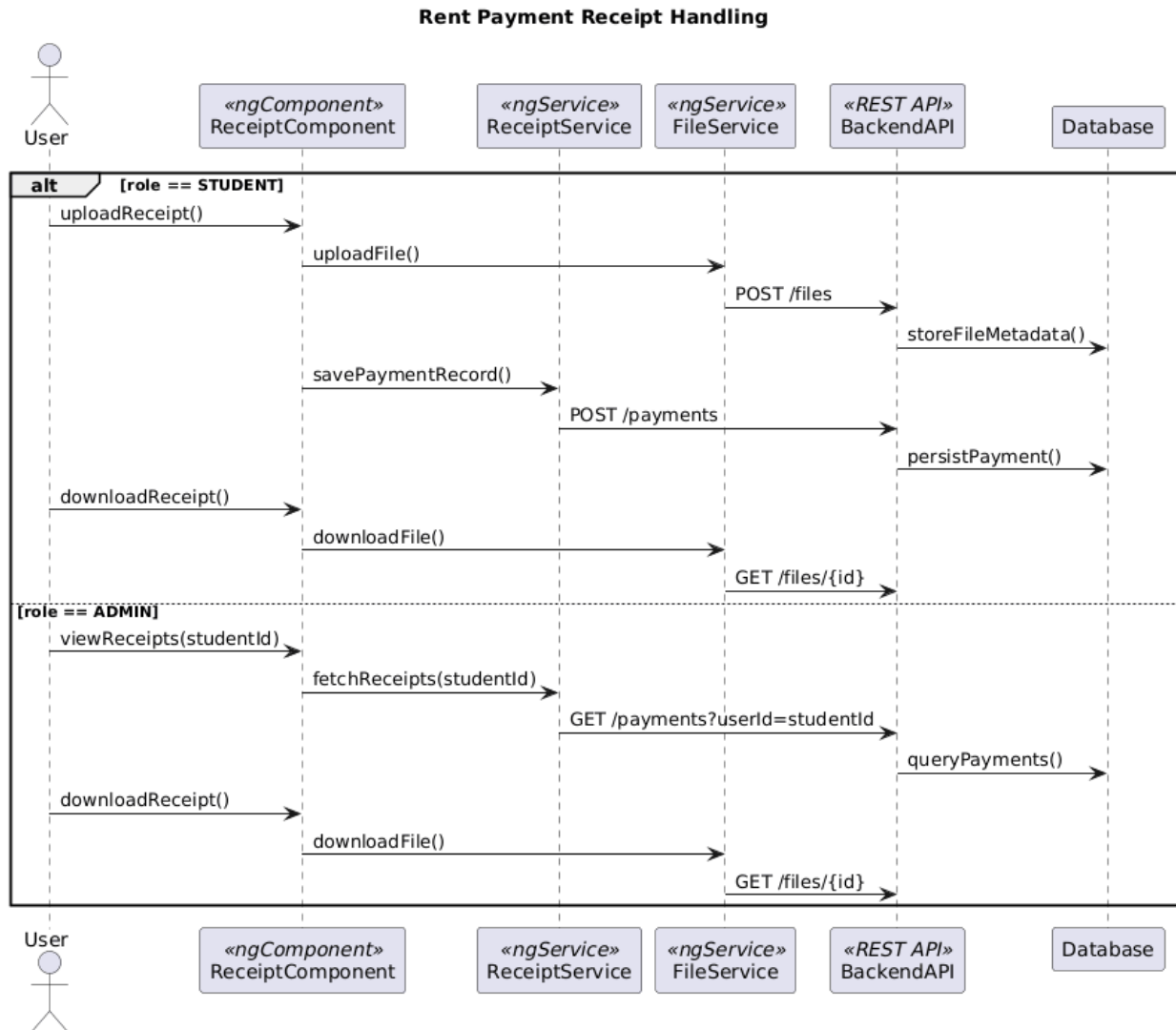


Figure 7: Rent Payment Receipt Upload & Download Sequence Diagram

- A student uploads a rent payment receipt via the ReceiptComponent.
- The component delegates file handling to the FileService.
- The backend stores the receipt file and associated metadata.
- Rent payment records are created through the ReceiptService.
- Students can download their own receipts via the ReceiptComponent.
- Administrators can view and download receipts for selected students.

- **Rent Payment Report Generation with Progress Scenario**

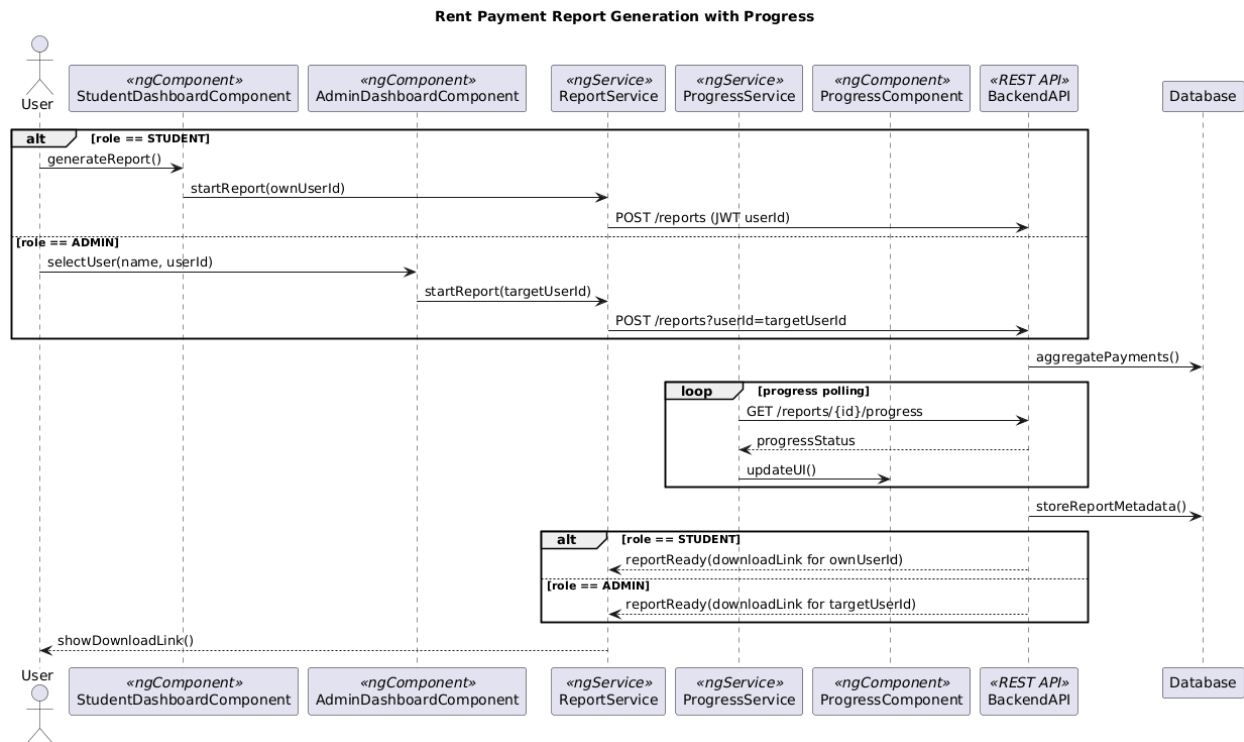


Figure 8: Rent Payment Report Generation with Progress Sequence Diagram

- A student or administrator initiates report generation from their respective dashboard.
- The request is forwarded to the ReportService.
- The backend starts a long-running report generation process.
- Progress information is periodically retrieved via the ProgressService.
- The ProgressComponent displays visual progress indicators to the user.
- Once completed, the generated report is made available for download.
- Students download reports for their own user account, while administrators can download reports for any selected user.

Backend Design

The backend is implemented using Node.js with Express.js to provide RESTful APIs that support the Angular frontend. The system uses a MySQL relational database for structured data storage and Google Firebase Storage for managing uploaded files such as contracts and rent payment receipts. The backend also supports asynchronous processing for long-running tasks.

- **Key Features**

- **Authentication and Authorization**

- Secure user authentication using JWT.
 - Role-based access control for students and dormitory administrators.

- **RESTful APIs**

- **User Management:** Login, authentication, and role validation.
 - **Application Management:** Submit, review, accept, or reject dormitory applications.
 - **Contract Management:** Create, extend, and terminate dormitory contracts.
 - **Complaint Management:** Submit complaints and update their status.
 - **File Management:** Upload and download documents using Google Firebase Storage.
 - **Report Management:** Generate rent payment history reports.

- **File Storage**

- User-uploaded documents (contracts and payment receipts) are stored in Google Firebase Storage.
 - File metadata (file path, owner, upload date) is stored in the MySQL database.
 - Secure access rules ensure that files are accessible only to authorized users.

- **Long-Running Backend Processing**

- Rent payment report generation is handled as an asynchronous backend task.
 - The backend aggregates rent payment records from MySQL and generates report files.
 - Generated reports are stored in Firebase Storage and made available for download.

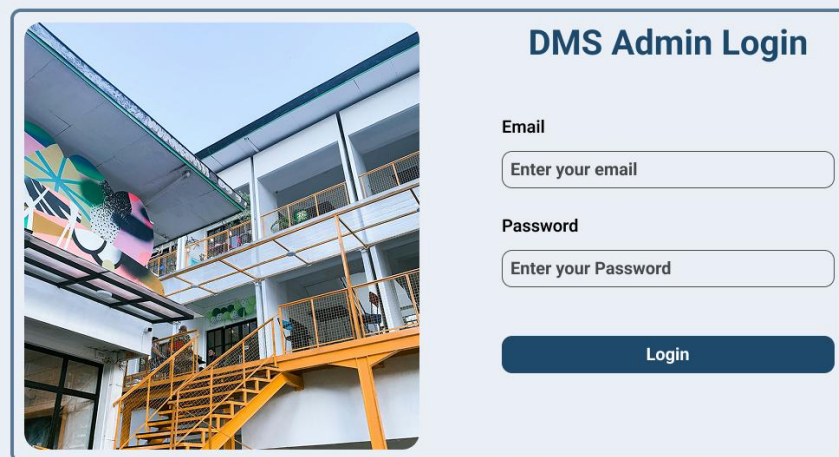
- **Progress Tracking**

- Backend services periodically update computation progress during report generation.
 - Progress information is exposed via APIs and consumed by the frontend to display visual progress indicators.

- **Database Schema Overview (MySQL)**

- **Users:** Stores user credentials, roles, and authentication data.
 - **StudentProfiles:** Stores student-specific profile and academic information.
 - **Applications:** Tracks dormitory application data and decision status.
 - **Contracts:** Manages dormitory contract lifecycle information.
 - **RentPayments:** Stores structured rent payment records.
 - **Complaints:** Logs complaints and resolution status.
 - **Reports:** Stores report metadata and references to generated files in Firebase Storage.
 - **Files:** Stores file metadata such as storage paths, ownership, and upload timestamps.

Screen Mocks



DMS Admin Login

Email

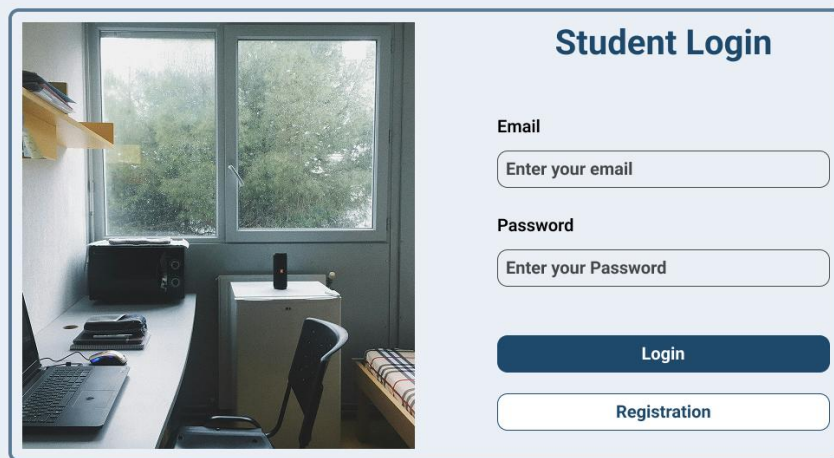
Enter your email

Password

Enter your Password

Login

Figure 9: Mock Screen - Admin Login



The mockup consists of a rectangular frame containing two main elements. On the left is a photograph of a desk with a laptop, a printer, and a window looking out onto trees. On the right is a login form titled 'Student Login' in bold dark blue text. The form includes two input fields: 'Email' with the placeholder 'Enter your email' and 'Password' with the placeholder 'Enter your Password'. Below these fields are two buttons: a dark blue 'Login' button and a white 'Registration' button with a dark blue border.

Student Login

Email

Password

Login

Registration

Figure 10: Mock Screen - Student Login

The image shows a mockup of a student registration form. The form is contained within a light blue rounded rectangle with a dark blue border. At the top, the title "Student Registration" is centered in a bold, dark blue font. Below the title, there are four input fields, each preceded by a label in a bold, dark blue font: "Name", "Email", "Password", and "Re-Password". The input fields are light blue with rounded corners and a thin dark blue border. The first three fields have placeholder text: "Enter your Full Name", "Enter your Email", and "Enter your Password". The fourth field has placeholder text: "ReEnter your Password". Below the input fields is a dark blue button with the word "Register" in white, bold, sans-serif font. The entire form is set against a light blue background.

Student Registration

Name
Enter your Full Name

Email
Enter your Email

Password
Enter your Password

Re-Password
ReEnter your Password

Register

Figure 11: Mock Screen - Student Registration

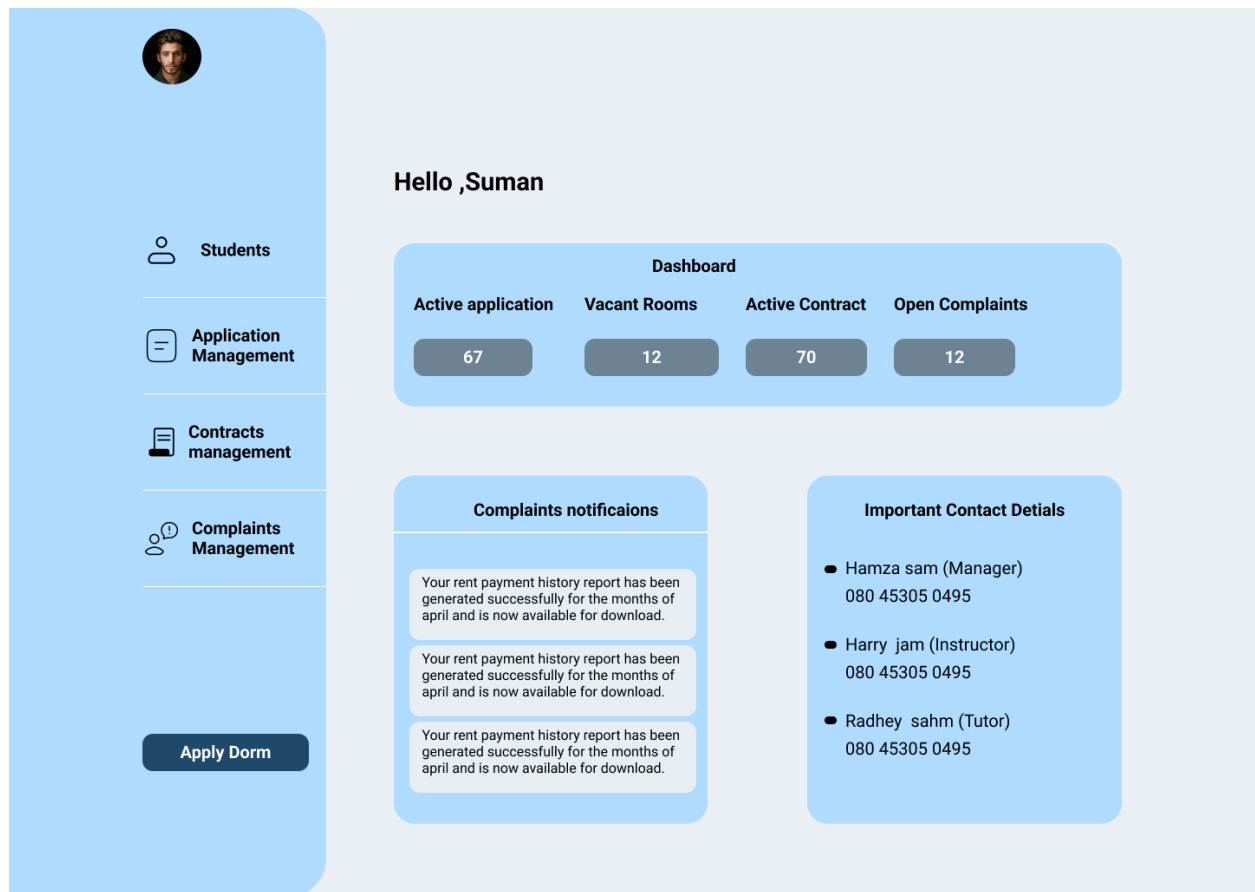


Figure 12: Mock Screen - Admin Dashboard

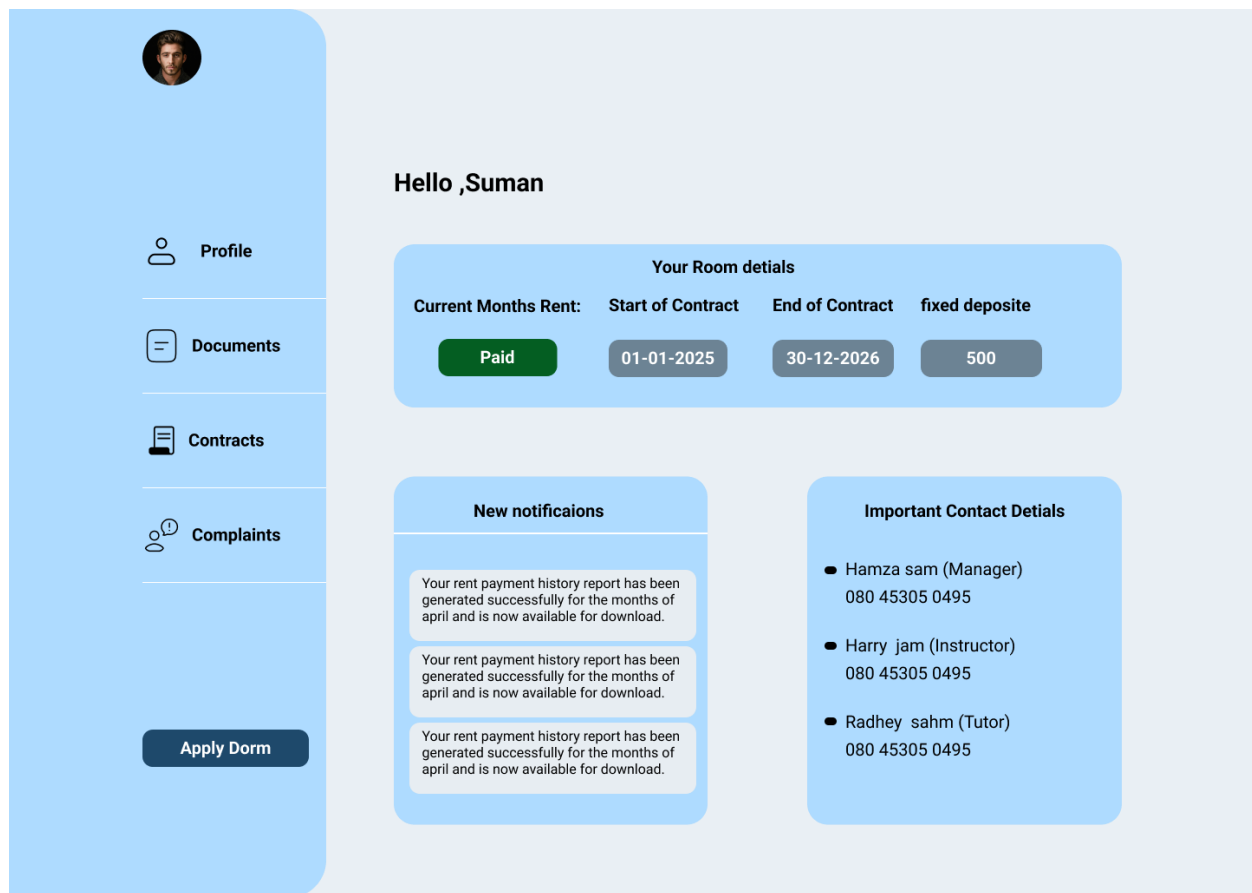


Figure 13: Mock Screen - Student Dashboard



 Profile

 Documents

 Contracts

 Complaints

Apply Dorm

Profile



Add New Photo

Name

Suman neupane

Email

abc@gmail.com

Mobile No

08034 50 4 345 5

Student ID

1751373

Course

Software Engineering

University

Hildesheim University

Save

Figure 14: Mock Screen - Student Profile

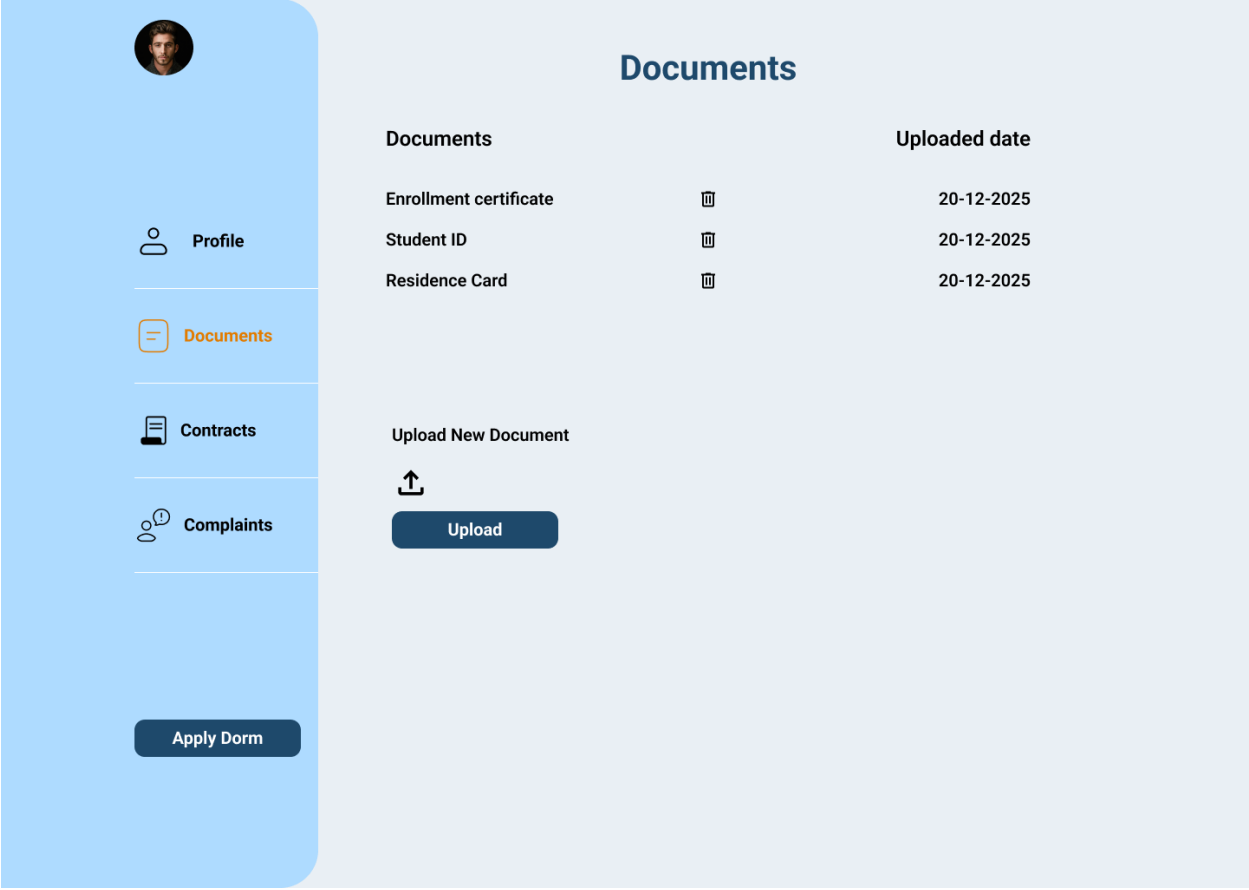


Figure 15: Mock Screen - Student Document

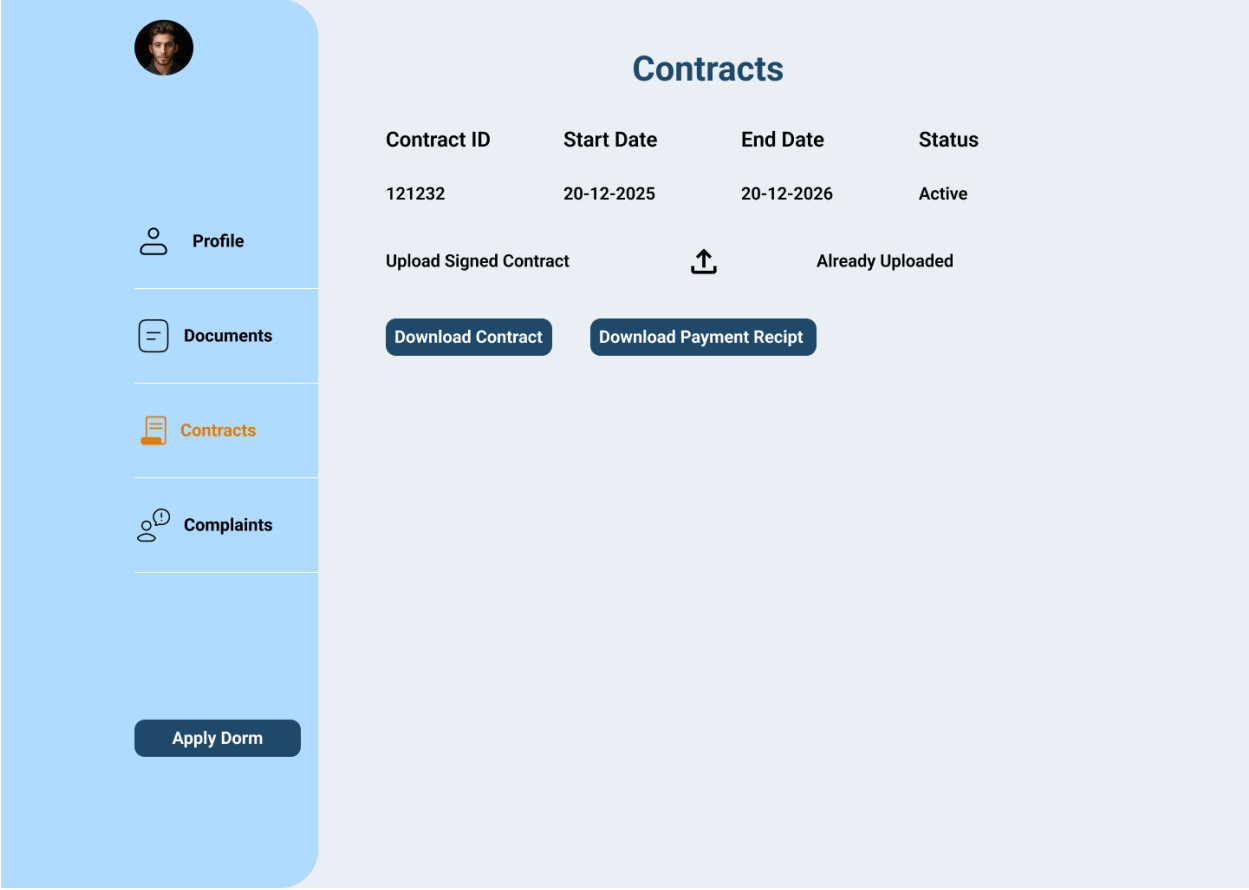


Figure 16: Mock Screen - Student Contracts



 Profile

 Documents

 Contracts

 Complaints

Apply Dorm

Past Complaints

Make New Complaints

Title

Enter the title of Complaints

Enter Your Complaints

Enter the your Complaints here.....

Submit

Figure 17: Mock Screen - Student Complaints



 Profile

 Documents

 Contracts

 Complaints

Apply Dorm

Apply



Name

Suman neupane

Email

abc@gmail.com

Mobile No

08034 50 4 345 5

Student ID

1751373

Course

Software Engineering

University

Hildesheim University

Room Preference

droop down options

Upload Student ID

Upload Enrollment

Upload Passport

Start Date

01-03-2026

☐ I confirm that the information provided is correct.

Apply Dorm

Figure 18: Mock Screen - Student Apply Dorm